
ABSOLUTE RADIANCE

Raytracing and Parallel Compute



SEDS CELESTIA · SIMULATIONS

INDUCTIONS 2026 · WEEK 3

Due: EOD, 30th September 2026

Prerequisites: Week 2 (Cellular Automata), Basic Python

This assignment concludes your induction track. You are expected to attempt all milestones and submit the Google Form with your GitHub repository linked. **Along with the code, write a short assignment brief in your repository's README.md: answer the two questions marked in the text, and describe your final render.**

Introduction. Last week you simulated sand falling through a discrete grid, utilizing a stepping algorithm to traverse an array. This week, we abandon the grid entirely. We are moving into continuous 3D space, and we are handing the math over to the GPU.

In the Workshop we covered the fundamentals of shaders, graphics programming and bits of raytracing. The architecture we will build is primitive by design, making it straightforward to build and understand over a single weekend, while still delivering stunning, physically accurate results.

To achieve real-time performance, you will write this in a **Fragment Shader** using GLSL (OpenGL Shading Language). A fragment shader is a program that executes simultaneously on thousands of GPU cores, calculating the physics for every single pixel on your screen at the exact same time. The **Vertex Shader** is already written for your convenience.

The starter template provides a `main.py` host application. It manages the Pygame window and sends the screen coordinates to the graphics card. You do not need to edit it. Your entire assignment takes place inside `shader.frag`.

A NOTE ON VECTOR MATH

A raytracer is built entirely on 3D vectors. The template provides several helper functions so you do not have to write them from scratch:

The Parametric Ray. A ray is a line in 3D space. It has an origin **O** and a direction

d. We can find any point P along the ray at a given time t using the function `ray_at()`:

$$P(t) = \mathbf{O} + t\mathbf{d}$$

Dot Product. The dot product of two vectors tells us how aligned they are. If you dot a vector with itself ($\mathbf{v} \cdot \mathbf{v}$), you get its squared length. If you dot two perpendicular vectors, you get exactly 0.

I. THE RAY AND THE SPHERE

Shooting Rays. For every pixel on the screen, your camera shoots a ray out into the world. To draw a sphere, we must ask: does this ray ever intersect the sphere's surface?

Mathematically, a sphere of radius R centered at $\mathbf{C}$ is the set of all points P where $(P - \mathbf{C}) \cdot (P - \mathbf{C}) = R^2$. If we substitute our ray equation into this sphere equation, we get a quadratic equation in terms of t :

$$(\mathbf{d} \cdot \mathbf{d})t^2 + 2\mathbf{d} \cdot (\mathbf{O} - \mathbf{C})t + (\mathbf{O} - \mathbf{C}) \cdot (\mathbf{O} - \mathbf{C}) - R^2 = 0$$

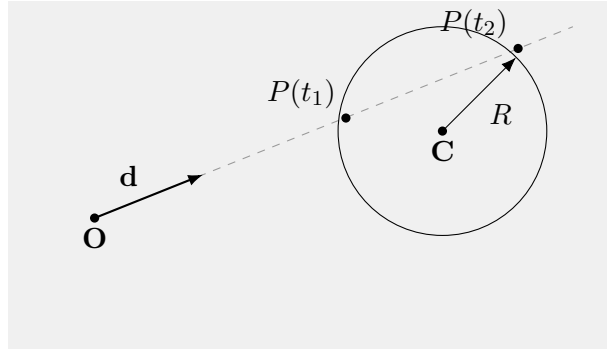


Figure 1. Ray-Sphere intersection. Solving the quadratic equation yields zero, one, or two solutions for t . We only care about the smallest positive t (the closest hit).

Your first task is to implement this quadratic solver inside the `hit_sphere()` function. Calculate the discriminant ($b^2 - 4ac$). If it is negative, the ray missed. If it is positive, calculate the closest root t . Once you find t , use `ray_at()` to find the hit point P , and calculate the outward normal vector (the direction the surface is facing, calculated as $\frac{P - \mathbf{C}}{R}$).

II. THE WORLD OF OBJECTS

Finding out if a ray hits a single sphere is not enough. If we have multiple objects in our scene, a ray might intersect several of them. We only want to draw the object that is *closest* to the camera.

Tracking the Closest Hit. In your template, you will find a `hit_world()` function. Your second task is to loop through (or sequentially check) the hardcoded spheres in the scene. For each sphere, call your `hit_sphere()` logic. You must maintain a

`closest_so_far` variable. Every time you find a valid hit, update `closest_so_far` so that subsequent checks only return true if they are even closer to the camera. If the ray passes through the entire world without hitting anything, return false so the sky gradient can be drawn.

III. LIGHT TRANSPORT AND MATERIALS

Once a ray hits a surface, it bounces. This is where a raytracer creates its signature photorealistic lighting. In the `ray_color()` function, you will implement an iterative loop that tracks the ray as it bounces up to 4 times, applying attenuation based on two material types:

Lambertian (Diffuse). Matte surfaces like chalk or rubber scatter light randomly. When a ray hits a Lambertian surface, it bounces off in a random direction within a hemisphere above the normal. We simulate this by taking the hit point, adding the surface normal, and adding a random unit vector.

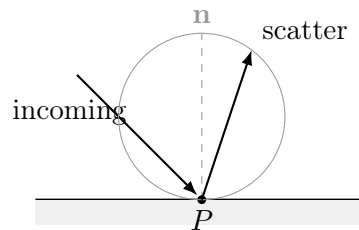


Figure 2. Lambertian scattering. The new target direction is calculated as $P + \mathbf{n} + \text{random_unit_vector}()$.

Metal (Reflective). Smooth surfaces bounce light perfectly, like a mirror. You calculate the reflected ray using the formula:

$$\mathbf{v}_{\text{reflected}} = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n}$$

To make the metal look frosted or "fuzzy", add a small random unit vector multiplied by a `fuzz` parameter before saving the new direction.

Question 1. In the `ray_color` loop, you must ensure rays do not intersect at exactly $t = 0.0$, but rather at a small offset like $t = 0.001$. If we set the minimum distance to 0.0, the image becomes covered in dark noise known as "shadow acne." Why does this happen? (Hint: Think about floating-point rounding errors when a ray bounces off a surface).

Question 2. In many CPU implementations of raytracing, the `ray_color` function calls itself recursively every time it hits an object to calculate the next bounce. In our GLSL fragment shader, we use a `for` loop instead. Why can't we use standard recursive function calls inside a GPU shader?

DELIVERABLE. A working GPU raytracer. Your repository should contain:

- `shader.frag` with all 3 sections fully implemented.
- A `README.md` answering the two aside questions, alongside a brief explanation of how

you approached the assignment.

- A clean git history with descriptive commit messages.

IV. BONUS

OPTIONAL. Everything in this section is optional. Attempt it only once your basic diffuse and metallic spheres are rendering correctly.

Because your host application is passing `u_time` to the GPU, your scene does not have to be static.

- **Motion:** Make one of the spheres orbit the center by adding `sin(u_time)` and `cos(u_time)` to its *X* and *Z* coordinates inside `hit_world()`. Because the raytracer accumulates frames over time, fast-moving objects will automatically generate physically accurate motion blur!
- **New Geometry:** A sphere is easy because the math is simple. Can you research and implement a ray-plane intersection to replace the giant green ground sphere with an infinite flat floor?

RECOMMENDED RESOURCES

- **Ray Tracing in One Weekend:** The assignment follows this largely (you may skip chapters 12 and 13).
- **The Book of Shaders:** Clean and gentle introduction to shaders.
- **Introduction to Shader Coding:** A great video primer to get you comfortable with fragment shaders.
- **Inigo Quilez's Website:** Articles by the goat of shader coding.
- **GLSL Under the Hood (Optional):** Not required for the assignment, but if you want to learn how GLSL works under the hood, read these articles on the **OpenGL rendering pipeline** and **drawing a full-screen quad**.